

C++ EQUALIZATOR PLUG-IN: DOCUMENTATION (ENG)

Aitor Pitarch Fontcuberta

This document's main objective is to serve as a detailed technical record of the design, architecture, and development process of a professional stereo parametric equalizer, created using the C++ programming language and the JUCE framework. The text aims to document how the technical challenges inherent in real-time audio programming were overcome, ensuring seamless and reliable communication between the processing engine and the user interface.

Regarding its content, the document describes the implementation of an architecture based on the JUCE State Management System (APVTS) to centralize the control of parameters such as frequencies, gains, and quality factors. It details the configuration of a DSP (Digital Signal Processing) chain that uses IIR filters and Butterworth designs to achieve precision cutoffs of up to 48 dB/oct on independent stereo channels. Furthermore, it addresses optimization techniques using C++ templates to avoid code duplication and efficient CPU management through the dynamic shutdown of unused nodes.

Finally, the document details the creation of the plugin's visual infrastructure, including the design of custom controls and the development of a spectrum analyzer based on the Fast Fourier Transform (FFT). It also documents the implementation of an interactive interface that allows the user to visualize the frequency response curve and manipulate audio parameters directly with the mouse over the graph, thus integrating visual and auditory feedback into a professional-grade tool.

1. Definition of the Parameter Layout (APVTS)

The basic architecture of the plugin controls has been implemented, using JUCE's state management system to centralize communication between the processor and the user interface.

```

186     juce::AudioProcessorValueTreeState::ParameterLayout EQAudioProcessor::createParameterLayout()
187     {
188     }
189     juce::AudioProcessorValueTreeState::ParameterLayout layout;
190
191     layout.add(std::make_unique<juce::AudioParameterFloat>("LowCut Frequency",
192     "Low Cut Frequency",
193     juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
194     20.0f));
195
196     layout.add(std::make_unique<juce::AudioParameterFloat>("HighCut Frequency",
197     "HighCut Frequency",
198     juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
199     20000.0f));
200
201     layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Frequency",
202     "Peak Frequency",
203     juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
204     750.0f));
205
206     layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Gain",
207     "Peak Gain",
208     juce::NormalisableRange<float>(-24.0f, 24.0f, 0.5f, 1.0f),
209     0.0f));
210
211     layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Quality",
212     "Peak Quality",
213     juce::NormalisableRange<float>(0.1f, 10.0f, 0.05f, 1.0f),
214     1.0f));
215
216     juce::StringArray stringArray;
217     for (int i = 0; i < 4; ++i)
218     {
219         juce::String str;
220         str << (12 + i * 12);
221         str << " dB/Oct";
222         stringArray.add(str);
223     }
224
225     layout.add(std::make_unique<juce::AudioParameterChoice>("LowCut Slope", "LowCut Slope", stringArray, 0));
226     layout.add(std::make_unique<juce::AudioParameterChoice>("HighCut Slope", "HighCut Slope", stringArray, 0));
227
228     return layout;
  
```

Parameters and Logic:

- Frequency Bands: Three control points (LowCut, HighCut, and Peak) are defined with a range of 20 Hz to 20 kHz. A resolution of 1.0 Hz is used.
- Gain Control (Peak): Range of ± 24 dB in 0.5 dB steps, allowing detailed control over the amplitude of the parametric band.
- Quality Factor (Q): Range from 0.1 to 10.0 in 0.05 steps, defining the bandwidth of the bell filter.
- Cutoff Slopes: Implementation of a discrete selection logic using a for loop that generates options of 12, 24, 36 and 48 dB/Oct for the high pass and low pass filters.

Functions and Variables Used:

- *createParameterLayout()* : Method responsible for instantiating and returning the ParameterLayout object necessary for the initialization of the AudioProcessorValueTreeState.
- *juce::AudioParameterFloat* : Class for continuous value parameters.
 - parameterID: Unique identifier string for automation.
 - parameterName: User-readable name in the DAW.
 - NormalisableRange<float>: Object that defines the behavior of the slider (min, max, interval, skew).
 - defaultValue: Initial value assigned when loading the plugin.

- `juce::AudioParameterChoice` : Class for multiple selection parameters (drop-down menus).
 - `choices`: StringArray that contains the text labels of the available options.
 - `defaultIndex` : Index of the option selected by default.
- `juce::NormalisableRange<float>` :
 - `rangeStart` / `rangeEnd`: Parameter limits.
 - `intervalValue`: Minimum increment of the value.
 - `skewFactor`: Skewness factor (1.0 for linear response)

The APVTS structure enables thread-safe parameter management, essential for synchronization between the audio and GUI threads. The use of NormalizableRange with specific steps ensures that the quantization of control values is consistent with the accuracy requirements of a professional signal processor.

2. Processing Chain (DSP Chain) Configuration

The signal processing infrastructure has been defined using JUCE's DSP module, structuring the filters into mono processing chains to manage stereo audio independently.

```

58
59 private:
60     using Filter = juce::dsp::IIR::Filter<float>;
61
62     using CutFilter = juce::dsp::ProcessorChain<Filter, Filter, Filter, Filter>;
63
64     using MonoChain = juce::dsp::ProcessorChain<CutFilter, Filter, CutFilter>;
65
66     MonoChain leftChain, rightChain;
67

```

Parameters and Logic:

- Filter Hierarchy: An alias Filter is used for single-precision IIR filters.
- CutFilter Structure: A ProcessorChain composed of 4 filters in series is defined. This architecture allows for a sum of 12 dB/Oct slopes for each filter, achieving the cutoffs of up to 48 dB/Oct defined in APVTS.
- Mono Chain (MonoChain): Final structure that organizes the signal flow in three stages: LowCut (CutFilter), Peak (Filter) and HighCut (CutFilter).
- Stereo Instantiation: Two instances of MonoChain (leftChain and rightChain) are declared to process the left and right channels separately.

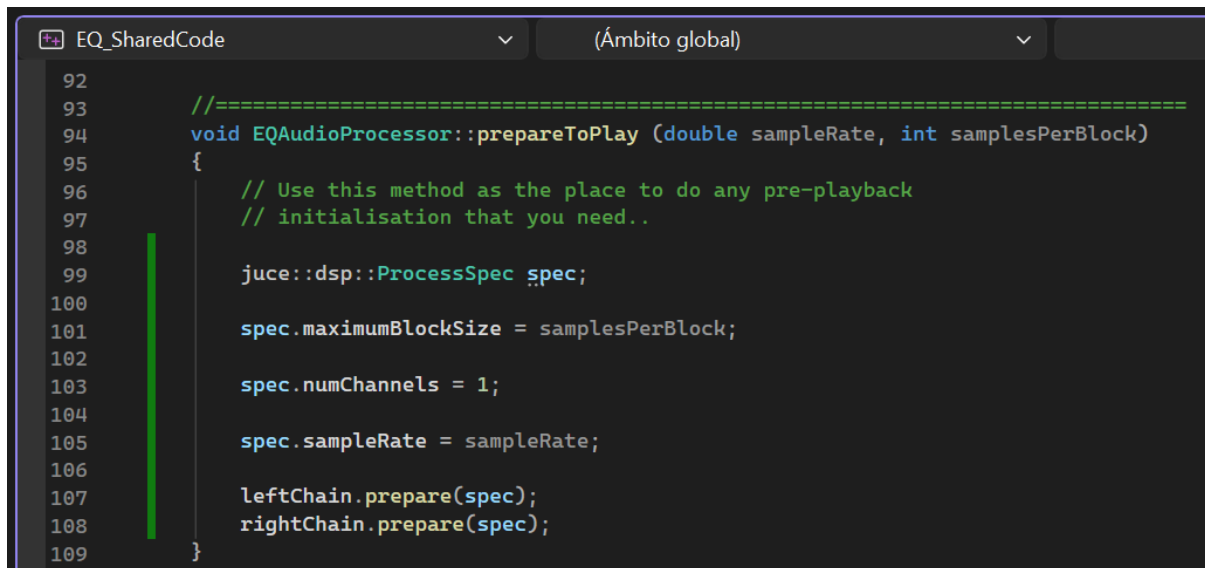
Functions and Variables Used:

- `juce::dsp::IIR::Filter<float>` : Base class for infinite impulse response filters.
- `juce::dsp::ProcessorChain` : Class template that allows chaining multiple DSP processors, automatically managing the passing of samples between them.
- `using (Type Aliasing)` : Used to simplify the declaration of complex template types, improving code readability.

The implementation establishes a serial filter topology. ProcessorChain's modularity allows each band to be treated as a logical unit, facilitating the expansion of the cutoff filter slope by concatenating second-order IIR instances.

3. Initialization in prepareToPlay

The DSP engine preparation has been implemented, ensuring that all components of the audio chain know the environment specifications before playback begins.



```
EQ_SharedCode (Ámbito global)
92
93 //=====
94 void EQAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
95 {
96     // Use this method as the place to do any pre-playback
97     // initialisation that you need..
98
99     juce::dsp::ProcessSpec spec;
100
101     spec.maximumBlockSize = samplesPerBlock;
102
103     spec.numChannels = 1;
104
105     spec.sampleRate = sampleRate;
106
107     leftChain.prepare(spec);
108     rightChain.prepare(spec);
109 }
```

Parameters and Logic:

- DSP configuration: A ProcessSpec object is used to pass the sampling rate and maximum block size to the filtering chains.
- Channel Synchronization: Each MonoChain is initialized with a single channel (numChannels = 1), as stereo processing is divided into two independent mono chains.

Functions and Variables Used:

- *prepareToPlay()* : AudioProcessor method that the DAW calls before starting audio processing.
- *juce::dsp::ProcessSpec* : Structure that contains:
 - *sampleRate*: Current sampling frequency (e.g. 44100 Hz).
 - *maximumBlockSize*: Maximum size of the audio blocks that the plugin will process.
 - *numChannels* : Number of channels that this specific processor will handle.
- *prepare()* : Method that allocates memory and precalculates necessary coefficients in each element of the DSP chain.

Before the music starts playing, the program needs to know the audio sample rate and how much information it will receive at once. This function gives the equalizer these instructions so it's ready and there are no errors or strange noises when you press "play."

4. Implementation of the Processing Loop (processBlock)

The core of real-time processing has been configured, where the incoming audio is broken down into mono blocks to be filtered by the corresponding DSP chains.

```
EQ_SharedCode
EQAudioProcessor
hasEditor() const

143 void EQAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
144 {
145     juce::ScopedNoDenormals noDenormals;
146     auto totalNumInputChannels = getTotalNumInputChannels();
147     auto totalNumOutputChannels = getTotalNumOutputChannels();
148
149     // In case we have more outputs than inputs, this code clears any output
150     // channels that didn't contain input data, (because these aren't
151     // guaranteed to be empty - they may contain garbage).
152     // This is here to avoid people getting screaming feedback
153     // when they first compile a plugin, but obviously you don't need to keep
154     // this code if your algorithm always overwrites all the output channels.
155     for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
156         buffer.clear (i, 0, buffer.getNumSamples());
157
158     juce::dsp::AudioBlock<float> block(buffer);
159
160     auto leftBlock = block.getSingleChannelBlock(0);
161     auto rightBlock = block.getSingleChannelBlock(1);
162
163     juce::dsp::ProcessContextReplacing<float> leftContext(leftBlock);
164     juce::dsp::ProcessContextReplacing<float> rightContext(rightBlock);
165
166     leftChain.process(leftContext);
167     rightChain.process(rightContext);
168 }
```

Parameters and Logic:

- Denormal Handling: ScopedNoDenormals is used to prevent CPU performance drops caused by denormal numbers (underflow) in recursive IIR filter calculations.
- Block Abstraction: The native AudioBuffer is converted into an AudioBlock, facilitating compatibility with the JUCE DSP module.
- Contextual Processing: Processing contexts are created for each channel, allowing filters to modify the audio directly "in-place".

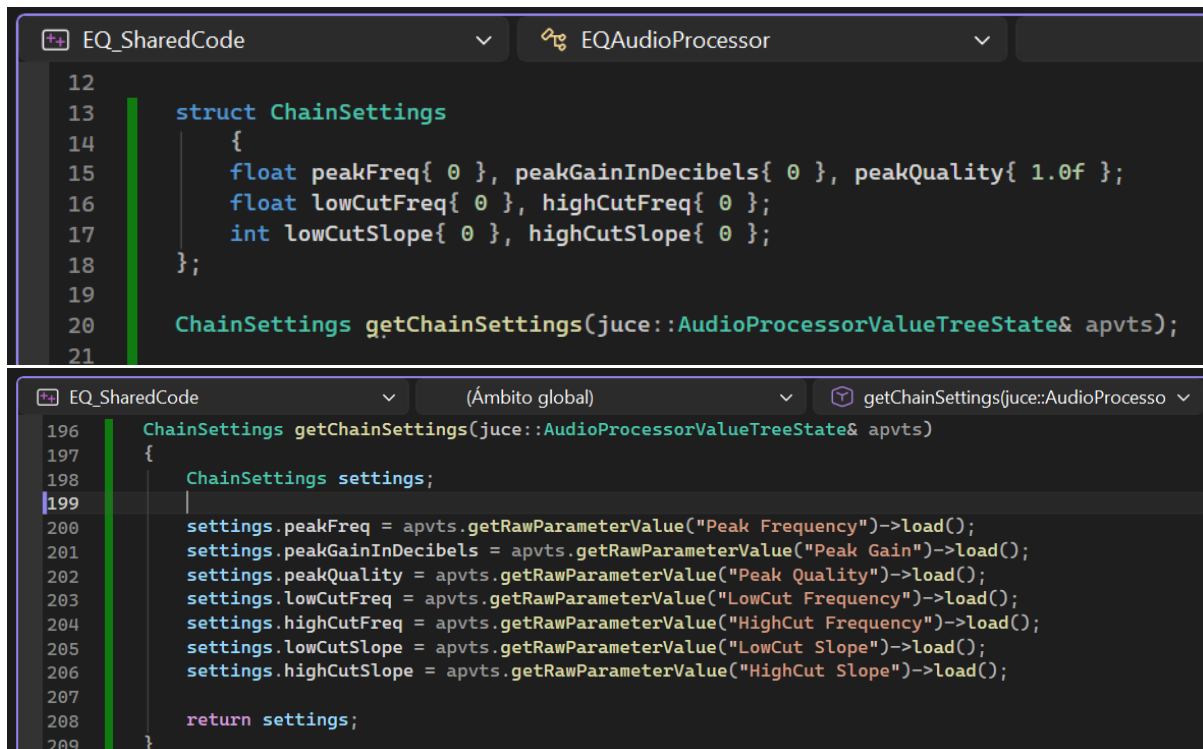
Functions and Variables Used:

- *juce::ScopedNoDenormals* : Class that optimizes CPU usage during processing by avoiding the handling of numbers extremely close to zero.
- *juce::dsp::AudioBlock<float>* : Class that wraps the audio buffer data.
 - *getSingleChannelBlock(index)* : Extracts a pointer and metadata from a specific channel block.
- *juce::dsp::ProcessContextReplacing<float>* : Context that manages the input and output of data on the same memory address, optimizing cache access.

The processing block segments the stereo data stream into independent mono streams. The use of replacement contexts ensures maximum efficiency by processing the filter coefficients directly on the input buffer, minimizing latency and memory overhead.

5. Data Structure and Parameter Extraction

An auxiliary structure and a utility function have been defined to centralize the obtaining of values from the `AudioProcessorValueTreeState`, facilitating its use in the audio loop.



```

12
13 struct ChainSettings
14 {
15     float peakFreq{ 0 }, peakGainInDecibels{ 0 }, peakQuality{ 1.0f };
16     float lowCutFreq{ 0 }, highCutFreq{ 0 };
17     int lowCutSlope{ 0 }, highCutSlope{ 0 };
18 };
19
20 ChainSettings getChainSettings(juce::AudioProcessorValueTreeState& apvts);
21

```

```

196 ChainSettings getChainSettings(juce::AudioProcessorValueTreeState& apvts)
197 {
198     ChainSettings settings;
199
200     settings.peakFreq = apvts.getRawParameterValue("Peak Frequency")->load();
201     settings.peakGainInDecibels = apvts.getRawParameterValue("Peak Gain")->load();
202     settings.peakQuality = apvts.getRawParameterValue("Peak Quality")->load();
203     settings.lowCutFreq = apvts.getRawParameterValue("LowCut Frequency")->load();
204     settings.highCutFreq = apvts.getRawParameterValue("HighCut Frequency")->load();
205     settings.lowCutSlope = apvts.getRawParameterValue("LowCut Slope")->load();
206     settings.highCutSlope = apvts.getRawParameterValue("HighCut Slope")->load();
207
208     return settings;
209 }

```

Parameters and Logic:

- ChainSettings Structure : Groups all control variables (frequencies, gains, qualities and slopes) into a single object to avoid repetitive and scattered calls to the APVTS.
- The `getChainSettings` function reads the current parameter values. It uses `load()` to ensure an atomic and safe read from the audio thread.
- Type Mapping: Frequencies and gains are extracted as float, while slope indices are retrieved as int.

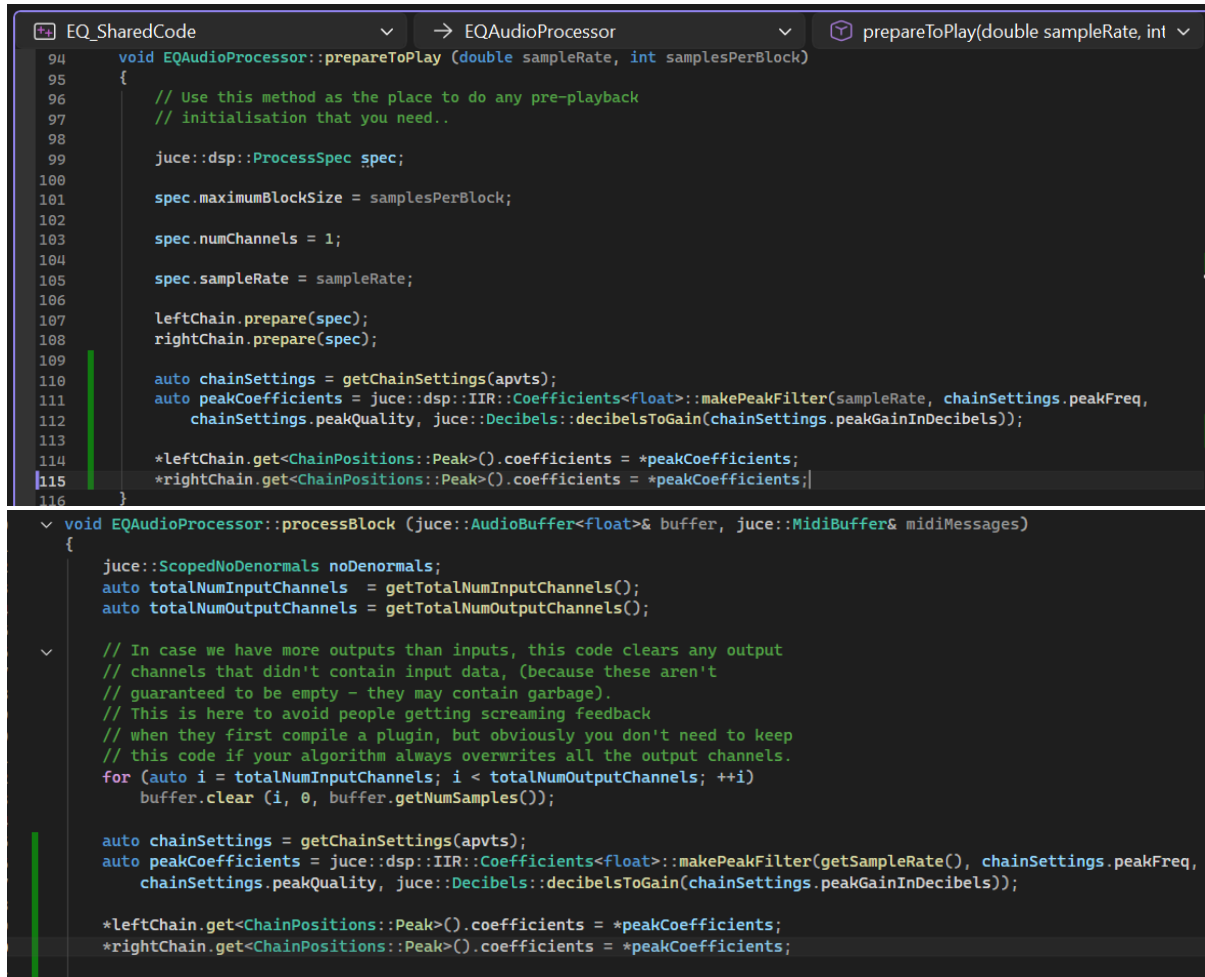
Functions and Variables Used:

- `apvts.getRawParameterValue()` : Gets a pointer to the real-time value of a parameter using its ID.
- `load()` : Method of `std::atomic` that ensures the value read is the most recent and consistent across different threads of execution.

Implementing a dedicated extraction function decouples the data retrieval logic from the processing logic. This improves maintainability and ensures that the audio thread works with a consistent "snapshot" of all controls at the start of each block.

6. Real-Time Coefficient Update

The necessary logic has been integrated to transform the control parameters into mathematical coefficients applicable to the Peak filter, both in the initialization and in the continuous process.



```

94 void EQAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
95 {
96     // Use this method as the place to do any pre-playback
97     // initialisation that you need..
98
99     juce::dsp::ProcessSpec spec;
100
101     spec.maximumBlockSize = samplesPerBlock;
102
103     spec.numChannels = 1;
104
105     spec.sampleRate = sampleRate;
106
107     leftChain.prepare(spec);
108     rightChain.prepare(spec);
109
110     auto chainSettings = getChainSettings(apvts);
111     auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(sampleRate, chainSettings.peakFreq,
112                                     chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));
113
114     *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
115     *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
116 }

```

```

void EQAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    // In case we have more outputs than inputs, this code clears any output
    // channels that didn't contain input data, (because these aren't
    // guaranteed to be empty - they may contain garbage).
    // This is here to avoid people getting screaming feedback
    // when they first compile a plugin, but obviously you don't need to keep
    // this code if your algorithm always overwrites all the output channels.
    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear(i, 0, buffer.getNumSamples());

    auto chainSettings = getChainSettings(apvts);
    auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(getSampleRate(), chainSettings.peakFreq,
        chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));

    *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
    *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
}

```

Parameters and Logic:

- Coefficient Calculation: The static function `makePeakFilter` is used to generate the coefficients of a bell filter based on the center frequency, Q factor, and gain.
- Unit Conversion: The decibel (dB) gain is transformed to linear gain using `decibelsToGain`, a necessary requirement for the DSP engine.
- Chain Assignment: The specific element of the processing chain (`ChainPositions::Peak`) is accessed and its coefficients are updated synchronously for the left and right channels.

Functions and Variables Used:

- `juce::dsp::IIR::Coefficients<float>::makePeakFilter` : Calculates the IIR filter coefficients based on:
 - `sampleRate`: Sampling frequency.
 - `frequency`: Central frequency of the bell.
 - `Q`: Quality factor (bandwidth).
 - `gain` : Linear gain.

- `leftChain.get<Index>()` : Template function that allows access to a specific processor within a ProcessorChain at compile time.
- `*coefficients = *newCoefficients` : Dereferencing operation to copy the values of the new coefficients into the pointer managed by the filter.

Updating coefficients in the processBlock allows the filter to respond instantly to changes in the interface. By replicating the same coefficients in leftChain and rightChain, identical phase and magnitude response is ensured in both channels, maintaining stereo imaging coherence.

7. Integration and Logic of the LowCut (High-Pass) Filter

LowCut filter processing has been implemented using a variable-order Butterworth filter topology. The filter slope determines how many links in the processing chain are activated to achieve the desired attenuation.

```

103     auto chainSettings = getChainSettings(apvts);
104     auto peakCoefficients = juce::dsp::IIR::Coefficients<Float>::makePeakFilter(sampleRate, chainSettings.peakFreq,
105                                     chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));
106
107     auto cutCoefficients = juce::dsp::FilterDesign<Float>::designIIRHighpassHighOrderButterworthMethod(chainSettings.lowCutFreq,
108                                                         getSampleRate(),
109                                                         2 * (chainSettings.lowCutSlope + 1));
110
111     *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
112     *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
113
114     // --- CONFIGURACIÓN LOW CUT IZQUIERDO ---
115     auto& leftLowCut = leftChain.get<ChainPositions::LowCut>();
116     leftLowCut.setBypassed<0>(true);
117     leftLowCut.setBypassed<1>(true);
118     leftLowCut.setBypassed<2>(true);
119     leftLowCut.setBypassed<3>(true);
120

```

```

switch (chainSettings.lowCutSlope)
{
case Slope::Slope_12:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    break;
}
case Slope::Slope_24:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    break;
}
case Slope::Slope_36:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    *leftLowCut.get<2>().coefficients = *cutCoefficients[2];
    leftLowCut.setBypassed<2>(false);
    break;
}
case Slope::Slope_48:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    *leftLowCut.get<2>().coefficients = *cutCoefficients[2];
    leftLowCut.setBypassed<2>(false);
    *leftLowCut.get<3>().coefficients = *cutCoefficients[3];
    leftLowCut.setBypassed<3>(false);
    break;
}
}

```

```

switch (chainSettings.lowCutSlope)
{
case Slope::Slope_12:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    break;
}
case Slope::Slope_24:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    break;
}
case Slope::Slope_36:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    *rightLowCut.get<2>().coefficients = *cutCoefficients[2];
    rightLowCut.setBypassed<2>(false);
    break;
}
case Slope::Slope_48:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    *rightLowCut.get<2>().coefficients = *cutCoefficients[2];
    rightLowCut.setBypassed<2>(false);
    *rightLowCut.get<3>().coefficients = *cutCoefficients[3];
    rightLowCut.setBypassed<3>(false);
    break;
}
}

```

Parameters and Logic:

- Butterworth Filter Design: Coefficients for a high-pass filter are generated based on the cutoff frequency (lowCutFreq) extracted from the APVTS.
- Filter Order Calculation: The function requires the mathematical filter order. Since each option in our menu (Slope_12, Slope_24, etc.) represents an index (0, 1, 2, 3), the formula $2 * (\text{chainSettings.lowCutSlope} + 1)$ translates this index into the correct orders (2, 4, 6, 8 respectively).
- Chain Management (Bypass): Our CutFilter consists of up to 4 individual filters (each contributing 12 dB/Oct). By default, all filters are bypassed (off) at the beginning of the block.

Functions and Variables Used:

- `juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod` : Returns a `ReferenceCountedArray` containing the coefficients for the various cascaded IIR filters needed to form the higher-order filter.
- `setBypassed<Index>(bool)` : Method of `juce::dsp::ProcessorChain` that activates (false) or deactivates (true) a specific processor within the chain without taking it out of the memory flow, saving CPU when steep slopes are not needed.
- Coefficient Dereferencing (`*cutCoefficients[i]`): Extracts the specific coefficient block from the generated array to assign it to position i of our LowCut substring.

Variable slope implementation is achieved by cascading multiple second-order (biquadratic) IIR filters. By using the Butterworth approximation, we guarantee the flattest possible frequency response in the passband (no ripple) at the expense of a nonlinear phase transition. Dynamically disabling unused nodes with `setBypassed` optimizes CPU cycles, ensuring that the audio engine only processes the mathematical operations strictly necessary for the selected slope (e.g., 1 filter for 12 dB/Oct, 4 filters for 48 dB/Oct).

8. Code Refactoring and Optimization

As the project scaled to support stereo processing (left and right channels) and preparations were made to add more filters (such as HighCut), the code in `prepareToPlay` and `processBlock` exhibited a high degree of duplication. Applying the DRY (Don't Repeat Yourself) principle, the filter update logic has been refactored using C++ templates in the header file (`PluginProcessor.h`).

```
using Coefficients = Filter::CoefficientsPtr;
static void updateCoefficients(Coefficients& old, const Coefficients& replacements);

template<int Index, typename ChainType, typename CoefficientType>
void update(ChainType& chain, const CoefficientType& coefficients)
{
    updateCoefficients(chain.template get<Index>().coefficients, coefficients[Index]);
    chain.template setBypassed<Index>(false);
}
```

```
template<typename ChainType, typename CoefficientType>
void updateCutFilter(ChainType& chain,
                    const CoefficientType& coefficients,
                    const Slope& slope)
{
    chain.template setBypassed<0>(true);
    chain.template setBypassed<1>(true);
    chain.template setBypassed<2>(true);
    chain.template setBypassed<3>(true);

    switch (slope)
    {
        case Slope_48:
        {
            update<3>(chain, coefficients);
        }
        case Slope_36:
        {
            update<2>(chain, coefficients);
        }
        case Slope_24:
        {
            update<1>(chain, coefficients);
        }
        case Slope_12:
        {
            update<0>(chain, coefficients);
        }
    }
}
```

New Helper Functions:

- `updateCoefficients` : A static function responsible for safely swapping old coefficients with new ones in memory.
- `update<Index, ChainType, CoefficientType>` : A template that abstracts the update of an individual link in the chain. It receives the filter index, assigns the corresponding coefficients, and activates it (taking it out of the bypass state).
- `updateCutFilter` : The core of the refactoring for cut filters. It groups together the logic that previously required massive, duplicated switch blocks for each channel.

Unlike a traditional switch statement where each case ends with a break, these have been deliberately omitted here. This means that if the user selects a slope of 48 dB/Oct (`Slope_48`), the code executes the update of the fourth filter (`update<3>`), and then cascades, also executing the 36, 24, and 12 dB/Oct cases.

9. Integration of the HighCut Filter and Completion of the DSP Chain

Once the optimal architecture was established using templates to avoid code duplication, the last link in our current equalization chain was integrated: the HighCut filter (also known as a low-pass filter).

```
void EQAudioProcessor::processBlock(juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear(i, 0, buffer.getNumSamples());

    //Peak update-----
    auto chainSettings = getChainSettings(apvts);
    updatePeakFilter(chainSettings);
    //-----

    //LowCut update-----
    auto cutCoefficients = juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod(chainSettings.lowCutFreq,
                                                                                                     getSampleRate(),
                                                                                                     2 * (chainSettings.lowCutSlope + 1));

    //LeftLowCut update
    auto& leftLowCut = leftChain.getChainPositions::LowCut();
    updateCutFilter(leftLowCut, cutCoefficients, chainSettings.lowCutSlope);

    //rightLowCut update
    auto& rightLowCut = rightChain.getChainPositions::LowCut();
    updateCutFilter(rightLowCut, cutCoefficients, chainSettings.lowCutSlope);
    //-----

    //HighCut update-----
    auto highCutCoefficients = juce::dsp::FilterDesign<float>::designIIRLowpassHighOrderButterworthMethod(chainSettings.highCutFreq,
                                                                                                       getSampleRate(),
                                                                                                       2 * (chainSettings.highCutSlope + 1));

    //LeftHighCut
    auto& leftHighCut = leftChain.getChainPositions::HighCut();
    updateCutFilter(leftHighCut, highCutCoefficients, chainSettings.highCutSlope);

    //RightHighCut
    auto& rightHighCut = rightChain.getChainPositions::HighCut();
    updateCutFilter(rightHighCut, highCutCoefficients, chainSettings.highCutSlope);
    //-----

    //Final audio processing-----
    juce::dsp::AudioBlock<float> block(buffer);

    auto leftBlock = block.getSingleChannelBlock(0);
    auto rightBlock = block.getSingleChannelBlock(1);

    juce::dsp::ProcessContextReplacing<float> leftContext(leftBlock);
    juce::dsp::ProcessContextReplacing<float> rightContext(rightBlock);

    leftChain.process(leftContext);
    rightChain.process(rightContext);
}
```

Evolution of prepareToPlay and processBlock:

1. Coefficient Generation: The coefficients for the three bands (*peakCoefficients* , *lowCutCoefficients* , and *highCutCoefficients*) are now calculated simultaneously. For the HighCut, the JUCE *designIIRLowpassHighOrderButterworthMethod* function is used , which, like the LowCut, requires the cutoff frequency, the sampling frequency, and the order of the mathematical filter calculated from the selected slope.
2. Simplified Node Updates: Instead of writing multiple switch blocks to evaluate which filters to turn on or off in each channel, the flow has been reduced to direct, semantic calls to our helper functions:
 - The Peak filter coefficients are updated directly using the *updateCoefficients* function .
 - *updateCutFilter* function is called , passing it the corresponding chain (leftChain or rightChain), the generated coefficients, and the slope selected by the user in the APVTS. Under the hood, this function handles all routing for active and bypassed nodes.
3. Clean Stereo Processing: Finally, the audio blocks from the left (0) and right (1) channels are extracted from the main buffer and passed through their respective processing chains (leftChain.process and rightChain.process).

The `prepareToPlay` method has been updated with this exact same structure. This ensures that when the audio engine starts or changes its hardware settings (such as the sample rate), all filters in all positions (LowCut, Peak, HighCut) are initialized with the correct coefficients before processing the first block of audio, preventing pops or unstable behavior.

10. Cleaning the processBlock with helper functions.

Although using templates in the previous step greatly improved filter routing, the processBlock method was still responsible for calculating the audio coefficients. Its sole purpose should be to process audio as quickly as possible. Therefore, we have extracted all the calculation and update logic into dedicated, semantic functions.

```
void EQAudioProcessor::updateLowCutFilters(const ChainSettings& chainSettings)
{
    auto cutCoefficients = juce::dsp::FilterDesign<Float>::designIIRHighpassHighOrderButterworthMethod(chainSettings.lowCutFreq,
                                                                                                     getSampleRate(),
                                                                                                     2 * (chainSettings.lowCutSlope + 1));

    auto& leftLowCut = leftChain.getChainPositions::LowCut();
    auto& rightLowCut = rightChain.getChainPositions::LowCut();
    updateCutFilter(leftLowCut, cutCoefficients, chainSettings.lowCutSlope);
    updateCutFilter(rightLowCut, cutCoefficients, chainSettings.lowCutSlope);
}

void EQAudioProcessor::updateHighCutFilters(const ChainSettings& chainSettings)
{
    auto highCutCoefficients = juce::dsp::FilterDesign<Float>::designIIRLowpassHighOrderButterworthMethod(chainSettings.highCutFreq,
                                                                                                     getSampleRate(),
                                                                                                     2 * (chainSettings.highCutSlope + 1));

    auto& leftHighCut = leftChain.getChainPositions::HighCut();
    auto& rightHighCut = rightChain.getChainPositions::HighCut();
    updateCutFilter(leftHighCut, highCutCoefficients, chainSettings.highCutSlope);
    updateCutFilter(rightHighCut, highCutCoefficients, chainSettings.highCutSlope);
}

void EQAudioProcessor::updateFilters()
{
    auto chainSettings = getChainSettings(apvts);
    updatePeakFilter(chainSettings);
    updateLowCutFilters(chainSettings);
    updateHighCutFilters(chainSettings);
}
```

New Control Functions (Helper Methods):

Specific methods have been implemented to isolate the logic of each band:

- *updatePeakFilter* : Gets the user parameters, calculates the IIR coefficients for the bell filter and applies them to both channels using updateCoefficients .
- *updateLowCutFilters* and *updateHighCutFilters* : They encapsulate the call to the Butterworth filter design function (dynamically calculating the order according to the slope) and pass the results to our updateCutFilter template to manage the stereo chain.
- *updateFilters* : A function that groups the calls to the previous functions, ensuring that the entire EQ is updated in a single step.

```
void EQAudioProcessor::processBlock(juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear(i, 0, buffer.getNumSamples());

    updateFilters();

    juce::dsp::AudioBlock<float> block(buffer);

    auto leftBlock = block.getSingleChannelBlock(0);
    auto rightBlock = block.getSingleChannelBlock(1);

    juce::dsp::ProcessContextReplacing<float> leftContext(leftBlock);
    juce::dsp::ProcessContextReplacing<float> rightContext(rightBlock);

    leftChain.process(leftContext);
    rightChain.process(rightContext);
}
```

By extracting all the mathematical logic into helper functions and grouping them under `updateFilters()`, we achieved an extremely clean `processBlock` focused exclusively on processing the audio, which is its true and sole responsibility. This abstraction completely eliminates code duplication between the processing block and `prepareToPlay`, greatly improving the project's readability and leaving it perfectly prepared to scale seamlessly if we decide to incorporate new equalization bands in the future.

11. Secure Communication Infrastructure: SingleChannelSampleFifo

To display the audio spectrum in the graphical interface without compromising signal processing performance, a circular FIFO (First-In, First-Out) buffer has been implemented. This structure allows audio samples to be extracted from the highest-priority thread in a completely safe and lock-free manner.

```
template<typename BlockType>
struct SingleChannelSampleFifo
{
    SingleChannelSampleFifo(Channel ch) : channelToUse(ch)
    {
        prepared.set(false);
    }

    // Llamado desde prepareToPlay (audio-thread, antes de procesar)
    void prepare(int bufferSize)
    {
        prepared.set(false);
        size.set(bufferSize);

        bufferToFill.setSize(1, // 1 canal (mono)
                             bufferSize, // keepExistingContent
                             false, // clearExtraSpace
                             true); // avoidReallocating

        abstractFifo.setTotalSize(bufferSize);
        prepared.set(true);
    }

    // Llamado desde processBlock - copia el canal indicado al FIFO
    void update(const BlockType& buffer)
    {
        jassert(prepared.get());
        jassert(buffer.getNumChannels() > channelToUse);

        auto* channelPtr = buffer.getReadPointer(channelToUse);

        int start1, size1, start2, size2;
        abstractFifo.prepareToWrite(buffer.getNumSamples(),
                                    start1, size1, start2, size2);

        if (size1 > 0)
            bufferToFill.copyFrom(0, start1, channelPtr, size1);
        if (size2 > 0)
            bufferToFill.copyFrom(0, start2, channelPtr + size1, size2);

        abstractFifo.finishedWrite(size1 + size2);
        numSamplesAvailable.set(abstractFifo.getNumReady());
    }
};
```

```
// Llamado desde la GUI - devuelve true si habia suficientes muestras
bool getNextAudioBlock(BlockType& destination)
{
    if (!prepared.get() ||
        abstractFifo.getNumReady() < destination.getNumSamples())
        return false;

    int start1, size1, start2, size2;
    abstractFifo.prepareToRead(destination.getNumSamples(),
                               start1, size1, start2, size2);

    if (size1 > 0)
        destination.copyFrom(0, 0, bufferToFill, 0, start1, size1);
    if (size2 > 0)
        destination.copyFrom(0, size1, bufferToFill, 0, start2, size2);

    abstractFifo.finishedRead(size1 + size2);
    numSamplesAvailable.set(abstractFifo.getNumReady());
    return true;
}

int getNumSamplesAvailable() const { return numSamplesAvailable.get(); }
bool isPrepared() const { return prepared.get(); }
int getSize() const { return size.get(); }

private:
    Channel channelToUse;
    juce::AbstractFifo abstractFifo { 1 };
    BlockType bufferToFill;
    juce::Atomic<bool> prepared { false };
    juce::Atomic<int> size { 0 };
    juce::Atomic<int> numSamplesAvailable { 0 };
};
```

Data Flow and Concurrency:

- **Lock-Free Management:** The juce::AbstractFifo class is used in conjunction with a static std::array to manage read and write indices. This ensures that the audio thread and the GUI thread never compete for the same memory block in a detrimental way.
- **Real-Time Extraction:** In the processor's processBlock method, once the audio has passed through the entire DSP filter chain, a copy of the processed samples from the left and right channels is sent to their respective FIFOs using the leftChannelFifo.push() and rightChannelFifo.push() methods.
- **Structural Decoupling:** The audio processor ("Backend") is limited to "pushing" data continuously, completely delegating the responsibility of extracting and processing that data to the graphical environment.

Using a lock-free structure is a critical design principle in the development of professional audio plugins. It avoids the use of mutexes or locks that could cause audio glitches if the processing thread had to wait for the graphical interface thread to write new data to the buffer.

12. Spectrum Analyzer (FFT) Engine

The SpectrumAnalyzer logic structure was created as the basis for visualizing the frequency spectrum of incoming/outgoing audio. This engine extracts the data from the aforementioned FIFO and converts it from the time domain to the frequency domain.

```
SpectrumAnalyzer(SingleChannelSampleFifo<juce::AudioBuffer<float>>& fifo)
: fifo(fifo),
  forwardFFT(FFT_ORDER),
  window(FFT_SIZE + 1, juce::dsp::WindowingFunction<float>::hann)
{
    fftData.assign(2 * FFT_SIZE, 0.0f);
    monoBuffer.resize(FFT_SIZE, 0.0f);
    drawingData.fill(0.0f);
}
```

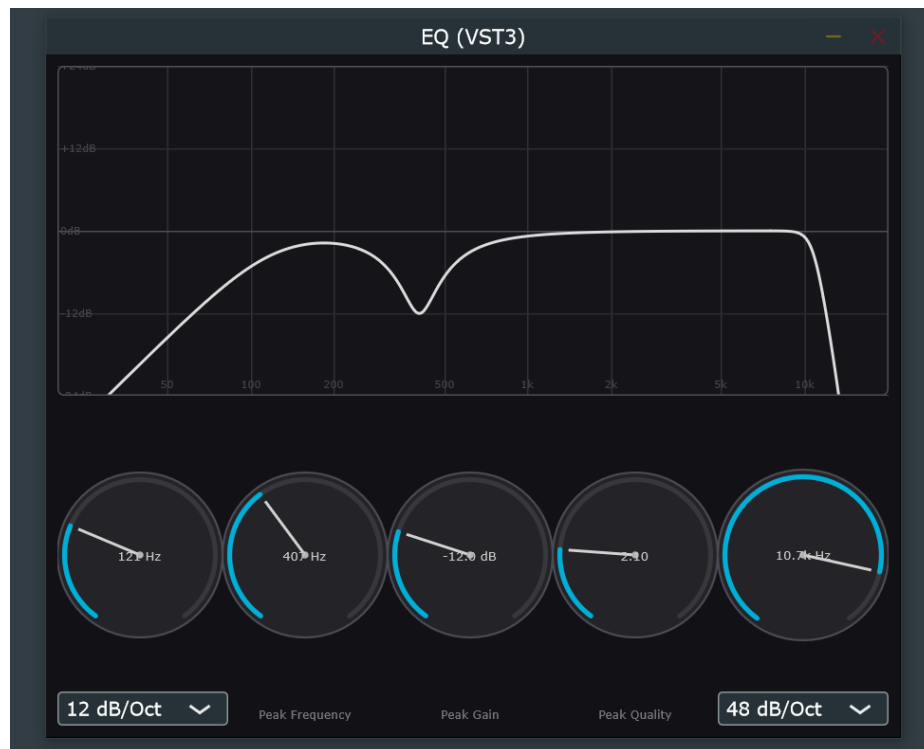
Processing on the Interface Thread:

- Fast Fourier Transform (FFT): The juce::dsp::FFT module is used . It has been configured with an "Order" of 11, which determines a window size of 2048 samples (2^{11}), offering high resolution of the low frequencies.
- Windowing: Before applying the FFT, the raw audio blocks are processed by a Hann-type windowing function (juce::dsp::WindowingFunction). This smooths the edges of the sample block, preventing mathematical imperfections (spectral leakage) that would generate false visual artifacts.
- Bin Generation: The process() method reads the accumulated samples, transforms them and calculates the resulting absolute values, normalizing them and preparing them in an internal array ready to be drawn on the screen.

Moving the FFT's computationally expensive calculations to the GUI thread ensures that the CPU dedicates its critical resources entirely to the process block. Currently, the FFT is not displayed in the plug-in and this will be fixed in a future update.

13. Visualization of the Response Curve

A component has been implemented that is dedicated exclusively to plotting the line representing how the filters are affecting the signal across the spectrum. To do this, instead of measuring the audio, the DSP filter chain is mathematically analyzed.



DSP Engine Simulation in the GUI

- Single-Channel Equalization Chain: The ResponseCurveComponent creates its own internal MonoChain. Upon initialization or when changes are detected (using parameterChanged), it updates this simulated chain with the same coefficients as the real processor .
- Logarithmic Magnitude Calculation: Using a juce::Timer running at 60 Hz, the component iterates through the pixel width of the screen, calculating the exact frequency corresponding to each pixel on a logarithmic scale (mapFreqToX function). It then requests the exact magnitude of the response at that frequency from the DSP engine using getMagnitudeForFrequency .
- Vector Graphics Generation: A juce::Path is drawn connecting the gain magnitudes obtained for each frequency, which draws a reliable, real-time representation of the equalization cuts and boosts.

The approach adopted allows separating the visual state from the actual processing state. By using parameter "Listener" implementations (juce::AudioProcessorParameter::Listener), the component ensures that the curve is only redrawn and recalculated when the user actually interacts with the interface or automates the values, guaranteeing highly efficient use of the graphics CPU.

14. Graphical Interface and Custom Controls

The default generic design has been replaced. Interactive physical controls (rotary sliders and menus) have been implemented that link bidirectionally with the AudioProcessorValueTreeState (APVTS).

```
void RotarySliderWithLabels::LookAndFeel::drawRotarySlider(
    juce::Graphics& g,
    int x, int y, int width, int height,
    float sliderPosProportional,
    float rotaryStartAngle,
    float rotaryEndAngle,
    juce::Slider& slider)
{
    using namespace juce;

    // Reservamos 16px abajo para el nombre del parámetro
    auto labelH = 16;
    auto bounds = Rectangle<float>(x, y, width, height - labelH);
    auto radius = jmin(bounds.getWidth(), bounds.getHeight()) / 2.0f;
    auto center = bounds.getCentre();

    // --- Fondo del knob ---
    g.setColour(Colour(35, 35, 40));
    g.fillEllipse(bounds.withSizeKeepingCentre(radius * 2.0f, radius * 2.0f));

    // --- Borde exterior ---
    g.setColour(Colour(75, 75, 85));
    g.drawEllipse(bounds.withSizeKeepingCentre(radius * 2.0f, radius * 2.0f), 1.5f);

    // --- Arco de recorrido total (gris oscuro) ---
    auto trackR = radius - 5.0f;
    Path arcBg;
    arcBg.addCentredArc(center.x, center.y, trackR, trackR,
        0.0f, rotaryStartAngle, rotaryEndAngle, true);
    g.setColour(Colour(55, 55, 60));
    g.strokePath(arcBg, PathStrokeType(3.5f, PathStrokeType::curved, PathStrokeType::rounded));

    // --- Arco de valor (azul-cian) ---
    float angle = rotaryStartAngle + sliderPosProportional * (rotaryEndAngle - rotaryStartAngle);
    Path arcVal;
    arcVal.addCentredArc(center.x, center.y, trackR, trackR,
        0.0f, rotaryStartAngle, angle, true);
    g.setColour(Colour(0, 175, 215));
    g.strokePath(arcVal, PathStrokeType(3.5f, PathStrokeType::curved, PathStrokeType::rounded));

    // --- Línea indicadora (desde centro hasta borde interior) ---
    auto thumbR = radius - 11.0f;
    Point<float> thumbPt(center.x + thumbR * std::sin(angle),
        center.y - thumbR * std::cos(angle));
    g.setColour(Colour(210, 210, 210));
    g.drawLine(Line<float>(center, thumbPt), 2.0f);

    // --- Punto central ---
    g.setColour(Colour(180, 180, 180));
    g.fillEllipse(Rectangle<float>(5.0f, 5.0f).withCentre(center));

    // --- Valor numérico (centrado en el knob) ---
    if (auto* rsw = dynamic_cast<RotarySliderWithLabels*>(&slider))
    {
        g.setColour(Colour(200, 200, 200));
        g.setFont(FontOptions(10.5f));
        g.drawFittedText(rsw->getDisplayString(),
            bounds.withSizeKeepingCentre(radius * 1.4f, radius * 0.6f).toNearestInt(),
            Justification::centred, 1);

        // --- Nombre del parámetro debajo del knob ---
        g.setColour(Colour(120, 120, 130));
        g.setFont(FontOptions(10.5f));
        auto labelBounds = Rectangle<int>(x, y + height - labelH, width, labelH);
        g.drawFittedText(rsw->param->getName(20), labelBounds, Justification::centred, 1);
    }
}
```

```

// --- Arco de valor (azul-cian) ---
float angle = rotaryStartAngle + sliderPosProportional * (rotaryEndAngle - rotaryStartAngle);
Path arcVal;
arcVal.addCentredArc(center.x, center.y, trackR, trackR,
    0.0f, rotaryStartAngle, angle, true);
g.setColour(Colour(0, 175, 215));
g.strokePath(arcVal, PathStrokeType(3.5f, PathStrokeType::curved, PathStrokeType::rounded));

// --- Línea indicadora (desde centro hasta borde interior) ---
auto thumbR = radius - 11.0f;
Point<float> thumbPt(center.x + thumbR * std::sin(angle),
    center.y - thumbR * std::cos(angle));
g.setColour(Colour(210, 210, 210));
g.drawLine(Line<float>(center, thumbPt), 2.0f);

// --- Punto central ---
g.setColour(Colour(180, 180, 180));
g.fillEllipse(Rectangle<float>(5.0f, 5.0f).withCentre(center));

// --- Valor numérico (centrado en el knob) ---
if (auto* rsw = dynamic_cast<RotarySliderWithLabels*>(&slider))
{
    g.setColour(Colour(200, 200, 200));
    g.setFont(FontOptions(10.5f));
    g.drawFittedText(rsw->getDisplayString(),
        bounds.withSizeKeepingCentre(radius * 1.4f, radius * 0.6f).toNearestInt(),
        Justification::centred, 1);

    // --- Nombre del parámetro debajo del knob ---
    g.setColour(Colour(120, 120, 130));
    g.setFont(FontOptions(10.5f));
    auto labelBounds = Rectangle<int>(x, y + height - labelH, width, labelH);
    g.drawFittedText(rsw->param->getName(20), labelBounds, Justification::centred, 1);
}
}
```

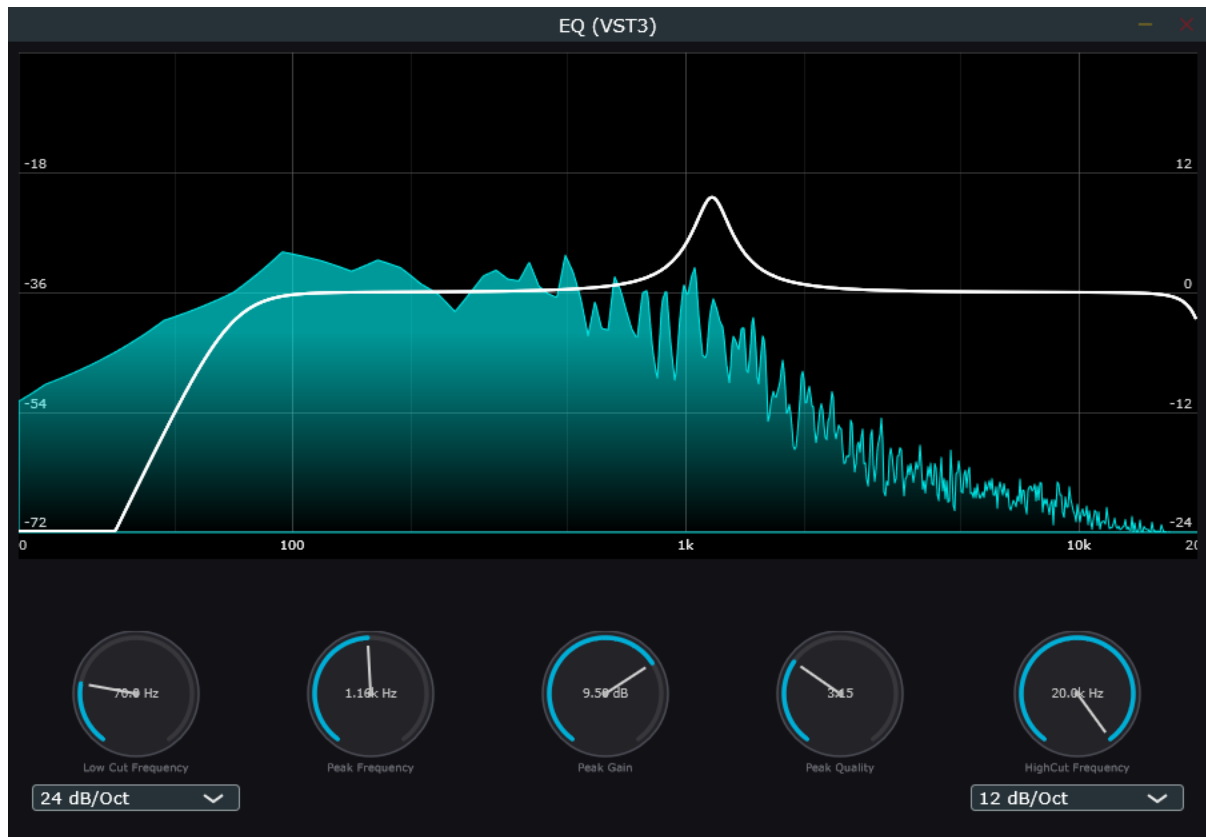
Aesthetics and Connection:

- Custom Components: The RotarySliderWithLabels class has been created, which inherits from juce::Slider. By overriding juce::LookAndFeel_V4, the drawRotarySlider method has been redefined to custom draw the potentiometer dial, position indicator, and numeric range labels.
- Use of Attachments: The SliderAttachment and ComboBoxAttachment objects have been instantiated. These ensure atomic synchronization between what the user sees in the interface and the actual values in the parameter state tree (APVTS).
- Proportional Layout Management: In the resized() method of the PluginEditor, areas are dynamically divided using removeFromTop and removeFromLeft, ensuring that the interface adapts gracefully without hardcoding absolute and fixed values.

Using the Attachment classes provided by JUCE prevents common concurrency and infinite feedback problems when turning a potentiometer. Furthermore, separating the drawing within a custom LookAndFeel class keeps the visual component class clean, promoting the reuse of design code in future plugins.

15. Visual Rendering of the Spectrum Analyzer (FFT)

With the data infrastructure (FIFOs) and the analyzer's logic engine already implemented in the processor, the graphical representation of the audio spectrum on the interface component has been completed in this phase.



Standardization and Layout:

- Constant Dynamic Range: In PluginEditor.h, the constants `NOISE_FLOOR = -72.0f` and `MAX_DB = 0.0f` have been defined within `SpectrumAnalyzer`. This sets the visual limits of the graph, ensuring that very low frequencies (background noise) are not plotted and that the strongest signal does not exceed the window.
- Mapping to the Y-axis: During the paint method, the FFT magnitude values (previously normalized between 0 and 1) are transformed into vertical coordinates. The value of 0 (or less than `NOISE_FLOOR`) is drawn at the base of the component, while the peaks are raised proportionally.

The FFT's visual rendering unifies auditory and visual feedback. By limiting the dynamic range with `NOISE_FLOOR` and processing the windowing (Hann window), "spectral clutter" is avoided, providing the user with a clean and professional reading of the signal's frequency content.

16. Interactive Curve Control (Mouse Management)

The ResponseCurveComponent has been transformed from a simple static viewer into an active control surface, allowing the user to modify the equalization by clicking and dragging directly on the graph.

```
static float mapXToFreq(float x, float leftX, float width)
{
    static const float logMin = std::log10(20.0f);
    static const float logMax = std::log10(20000.0f);
    float normalizedX = (x - leftX) / width;
    return std::pow(10.0f, logMin + normalizedX * (logMax - logMin));
}

static float mapFreqToX(float freq, float leftX, float width)
{
    // Eje X: log10(20..20000) → 0..width
    static const float logMin = std::log10(20.0f);
    static const float logMax = std::log10(20000.0f);
    float logFreq = std::log10(juce::jlimit(20.0f, 20000.0f, freq));
    return leftX + width * (logFreq - logMin) / (logMax - logMin);
}
```

Interaction Mathematics:

- Inverse Conversion (mapXToFreq): A mathematical utility function has been added that performs the reverse process of drawing. It takes the X coordinate of the pixel where the user clicked and converts it back to a frequency in Hertz (Hz) within the logarithmic scale of 20 Hz to 20 kHz.
- APVTS Update: When detecting mouse events (mouseDown, mouseDrag), the editor calculates the new frequency based on the cursor position and instantly updates the parameter value in the AudioProcessorValueTreeState.

Integrating mouse control directly into the graphic requires constant translation between the visual domain (linear pixels) and the audio domain (logarithmic frequencies). This interactivity dramatically improves the user experience (UX), allowing users to "touch" the sound instead of relying solely on the bottom knobs.

17. Layout and UX (User Experience) Refinement

Significant adjustments have been made to the PluginEditor's resized() method to ensure that the controls behave logically and aesthetically when the user resizes the plugin window.

```
void EQAudioProcessorEditor::resized()
{
    auto area = getLocalBounds().reduced(8);

    // 1. El visualizador ocupa el 65% superior
    auto displayArea = area.removeFromTop(juce::roundToInt(area.getHeight() * 0.65f));
    responseCurveComponent.setBounds(displayArea);

    // 2. Separación extra entre la gráfica y los controles
    area.removeFromTop(15);

    // 3. Forzamos una altura máxima para las ruedas para que no se vean gigantes
    auto controlsArea = area;
    int maxSliderHeight = 110;
    if (controlsArea.getHeight() > maxSliderHeight) {
        // Centramos el rectángulo de controles si nos sobra espacio vertical
        controlsArea = controlsArea.withSizeKeepingCentre(controlsArea.getWidth(), maxSliderHeight);
    }

    int sliderW = controlsArea.getWidth() / 5;

    // Distribuimos los sliders
    lowCutFreqSlider.setBounds(controlsArea.removeFromLeft(sliderW));
    peakFreqSlider.setBounds(controlsArea.removeFromLeft(sliderW));
    peakGainSlider.setBounds(controlsArea.removeFromLeft(sliderW));
    peakQualitySlider.setBounds(controlsArea.removeFromLeft(sliderW));
    highCutFreqSlider.setBounds(controlsArea);

    // 4. Ubicación de los comboboxes de Slope
    // Los colocamos justo debajo de sus respectivos controles de LowCut y HighCut
    int comboH = 20;

    // LowCut ComboBox debajo del LowCut Freq
    auto lowCutBounds = lowCutFreqSlider.getBounds();
    lowCutSlopeBox.setBounds(lowCutBounds.getX() + 10, lowCutBounds.getBottom() + 5, lowCutBounds.getWidth() - 20, comboH);

    // HighCut ComboBox debajo del HighCut Freq
    auto highCutBounds = highCutFreqSlider.getBounds();
    highCutSlopeBox.setBounds(highCutBounds.getX() + 10, highCutBounds.getBottom() + 5, highCutBounds.getWidth() - 20, comboH);
}
```

Proportion Control:

- Scale Limit (maxSliderHeight): A 110-pixel restriction has been introduced for the height of rotary controls (controlsArea.withSizeKeepingCentre). This prevents the undesirable behavior where the potentiometers became gigantic and pixelated when maximizing the window on large monitors.
- Semantic Grouping: The slope selectors (lowCutSlopeBox and highCutSlopeBox) are now dynamically positioned relative to their respective frequency potentiometers. Instead of occupying a portion of the overall layout, they are calculated using getBottom() from the slider area, keeping them visually grouped with the control they affect.

A truly responsive design in C++ involves more than just dividing screen percentages; it also involves defining logical limits (minWidth, maxHeight) to protect visual integrity. Spatially grouping dependent controls reinforces the user's mental model of the signal path.

18. Quick Status Management: "Default" Button (Parameter Reset)

To improve user experience (UX) and streamline workflow, a global "Default" button has been introduced. This button instantly returns all equalizer filters to their neutral position, saving the user the task of resetting the potentiometers one by one.

```
void EQAudioProcessorEditor::resetToDefaults()
{
    // Resetear los parámetros flotantes a sus valores por defecto
    audioProcessor.apvts.getParameter("LowCut Frequency")->setValueNotifyingHost(0.0f); // 20 Hz
    audioProcessor.apvts.getParameter("HighCut Frequency")->setValueNotifyingHost(1.0f); // 20000 Hz
    audioProcessor.apvts.getParameter("Peak Frequency")->setValueNotifyingHost(audioProcessor.apvts.getParameter("Peak Frequency")->getDefaultValue());
    audioProcessor.apvts.getParameter("Peak Gain")->setValueNotifyingHost(0.5f); // 0 dB (centro del rango)
    audioProcessor.apvts.getParameter("Peak Quality")->setValueNotifyingHost(audioProcessor.apvts.getParameter("Peak Quality")->getDefaultValue());

    // Resetear los slopes a 12 dB/Oct (índice 0)
    audioProcessor.apvts.getParameter("LowCut Slope")->setValueNotifyingHost(0.0f);
    audioProcessor.apvts.getParameter("HighCut Slope")->setValueNotifyingHost(0.0f);
}
```

Logic and Secure Synchronization:

- Interface Implementation: An instance of `juce::TextButton` has been added to `PluginEditor.h`. In the editor constructor, a lambda function is assigned to the `onClick` event, which triggers the `resetToDefaults()` method when the user clicks.
- Integration into the Layout: In the `resized()` method, a dedicated top strip of 20 pixels (`removeFromTop(20)`) has been reserved to house utilities, assigning the button to the top left corner of the control area.
- Atomic Synchronization (Host Notification): The most critical technical aspect of this implementation is the use of `setValueNotifyingHost(...)` on the APVTS parameters. Instead of visually changing the interface sliders, the code directly modifies the underlying state tree.

Using `setValueNotifyingHost()` ensures a perfect, thread-safe event cascade. When the button is pressed, the APVTS updates its internal value, immediately notifies the Attachment classes (causing the visual potentiometers to automatically move to their zero position), notifies the audio processor to recalculate the mathematical coefficients of the filters, and alerts the host (the DAW) to register the change at the automation level. All in a single step.

19. DSP Chain Expansion: 5-Band Parametric Equalizer

To offer greater versatility and control over the spectrum, the audio engine architecture has been expanded, moving from a basic design (LowCut, Peak, HighCut) to a complete parametric equalizer with three independent center bands.

```
// --- Tipos del procesador de filtros ---
using Filter = juce::dsp::IIR::Filter<float>;
using CutFilter = juce::dsp::ProcessorChain<Filter, Filter, Filter, Filter>;
using MonoChain = juce::dsp::ProcessorChain<CutFilter, Filter, Filter, Filter, CutFilter>;
MonoChain leftChain, rightChain;

private:

    enum ChainPositions { LowCut, Peak, Peak2, Peak3, HighCut };

    // --- Actualización de filtros ---
    void updatePeakFilter      (const ChainSettings& s);
    void updatePeak2Filter     (const ChainSettings& s);
    void updatePeak3Filter     (const ChainSettings& s);
    void updateLowCutFilters   (const ChainSettings& s);
    void updateHighCutFilters  (const ChainSettings& s);
    void updateFilters();
```

Architecture Expansion:

- MonoChain Modification: The main chain alias has been redefined to accommodate five links: `juce::dsp::ProcessorChain<CutFilter, Filter, Filter, Filter, CutFilter>` .
- New Parameters in APVTS: Six new floating parameters have been added to the state tree corresponding to the frequency, gain, and quality (Q) of Peak2 and Peak3 .
- Coefficient Processing: The calculation functions (`updatePeak2Filter` and `updatePeak3Filter`) have been replicated and adapted so that the processor independently feeds nodes 1, 2, and 3 of the chain with their respective coefficients.

Scaling the number of bands has been a clean and predictable process. Having previously invested in a `juce::dsp::ProcessorChain`-based architecture, adding new filters doesn't require complex routing mathematics; it's enough to define the link in the chain, register the parameter in APVTS, and send it the calculated coefficients, keeping the audio stream highly optimized.

20. State Management System: By-Band Bypass

A crucial feature for the mixing workflow has been implemented: the ability to turn each filter on and off (bypass) individually to listen to "A/B" comparisons without losing the parameter settings.

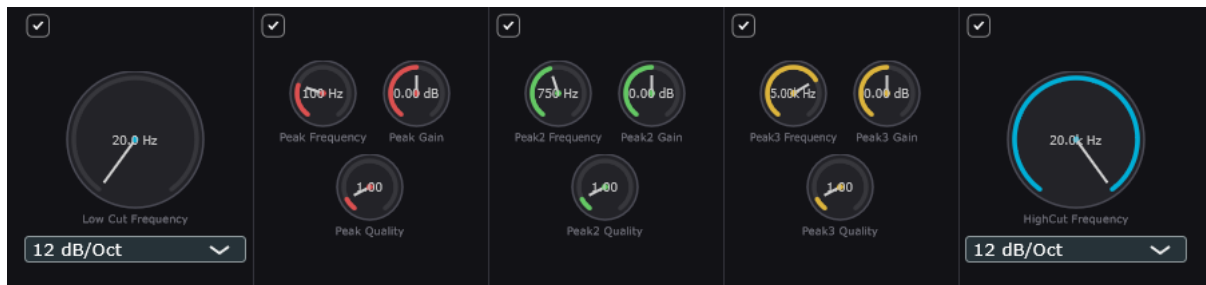
```
template<typename ChainType, typename CoefficientType>
void updateCutFilter(ChainType& chain,
                    const CoefficientType& coefficients,
                    const Slope& slope)
{
    chain.template setBypassed<0>(true);
    chain.template setBypassed<1>(true);
    chain.template setBypassed<2>(true);
    chain.template setBypassed<3>(true);
}
```

Safe Deactivation Logic:

- Boolean Parameters: Parameters of type `juce::AudioParameterBool` have been added to APVTS to represent the on/off state of each band (e.g., "Peak Bypass").
- DSP Integration: The `update` template function has been modified to receive the bypass status from `ChainSettings`. It uses the native `chain.template setBypassed<Index>(bypassed)` method, which tells the DSP engine to transparently skip that math node, also saving CPU cycles.

21. Evolution of the Graphical Interface: Multiband Layout and Visual Feedback

The PluginEditor has undergone a major restructuring to accommodate the new physical controls and visually reflect the activation status of each band.



Design and Usability (UX/UI):

- Layout Reorganization: The lower control area is now dynamically divided into 5 columns ($\text{sliderW} = \text{controlsArea.getWidth()} / 5$).
- Activation Buttons (juce::ToggleButton): A toggle button has been added to the top of each control column. These buttons communicate bidirectionally with the APVTS bypass booleans using ButtonAttachment.
- Visual Feedback for Bypass: The RotarySliderWithLabels class now detects if its filter is in bypass. If so, it reduces the opacity/color of the controls, visually indicating to the user that the filter is not currently affecting the audio signal.
-

22. Advanced Interactivity in the Curve Viewer

The ResponseCurveComponent has been enhanced to represent and allow interaction with the new parametric bands in an intuitive way.

Color Coding and Multi-Zone Interaction:

- Visual Differentiation: To avoid visual confusion, color constants (PEAK1_COLOR, PEAK2_COLOR, PEAK3_COLOR) have been defined. Each peak node is drawn with its respective color on the screen.
- Proximity Detection: During mouseDown and mouseDrag events, the algorithm now calculates the cursor's distance from the (X, Y) position of the three parametric nodes. The system automatically "hooks" and modifies the filter closest to the mouse, allowing for smooth graphic editing without needing to pre-select which band to adjust.

23. Release 1 Result:

Introduction: Release 2:

Release 2 represents a significant advancement in the interactivity and usability of the parametric equalizer. While Release 1 established a robust DSP processing architecture (LowCut, Peak, HighCut) with spectrum and response curve visualization, Release 2 focuses on transforming the response curve into an active and intuitive control surface.

24. Advanced Graphic Interactivity: Direct Editing on the Response Curve:

This is the most significant change in Release 2. The ResponseCurveComponent has been transformed from a simple static viewer into an active control surface that allows the user to edit all EQ parameters intuitively by dragging directly onto the frequency response graph.

Peak Detection and Hit-Testing

The system implements a proximity detection algorithm that continuously calculates the distance between the cursor position and each of the three peak points (Peak, Peak2, Peak3). When the user presses the mouse button within a selection radius (typically 30 pixels), the algorithm identifies the nearest peak as 'active' and enters edit mode.

This hit-test approach provides a forgiving capture area that makes editing smooth and comfortable, without requiring pinpoint accuracy. The three peaks are color-coded (Red, Green, Yellow), allowing for immediate visual identification of which one is being edited.

Logarithmic Frequency/Pixel Mapping

The critical logic of the system consists of two mapping functions that translate between the visual domain (linear pixels on the screen) and the audio domain (logarithmic frequencies from 20 Hz to 20 kHz):

- mapFreqToX(float freq): Converts a frequency in Hertz to an X coordinate in pixels
- mapXToFreq(float x): Performs the reverse conversion, transforming pixels to frequencies

Both functions use logarithmic arithmetic (base 10 log) to maintain the mathematically correct relationship between the on-screen display and the actual frequencies. This ensures that when the user drags the mouse an equal distance in pixels, the frequency change is consistent regardless of whether they are editing in the bass (20-200 Hz) or treble (5-20 kHz) range.

```
// Simplified example of logarithmic mapping
float mapFreqToX(float freq, float leftX, float width) {
    const float logMin = log10(20.0f);
    const float logMax = log10(20000.0f);
    float normalized = (log10(freq) - logMin) / (logMax - logMin);
    return leftX + normalized * width;
}

float mapXToFreq(float x, float leftX, float width) {
    const float logMin = log10(20.0f);
    const float logMax = log10(20000.0f);
    float normalized = (x - leftX) / width;
    return std::pow(10.0f, logMin + normalized * (logMax - logMin));
}
```

Parameter Editing: Freq, Gain and Q

During mouse drag, the system updates three independent parameters:

- Frequency (X): Horizontal mouse movement translates directly into frequency changes. The logarithmic scale ensures the user feels equal control across the entire spectrum.
- Gain (Y): Vertical movement is mapped to gain changes in decibels. Up = boost, down = cut. Sensitivity is constant in pixels/dB.
- Q-factor (Wheel): The mouse wheel adjusts the quality factor (Q) multiplicatively. With Ctrl pressed, the sensitivity is multiplied by 3 for quick changes. Without Ctrl, it's more precise.

Immediate spread to APVTS

Each parameter change is immediately propagated to the `AudioProcessorValueTreeState` using `setValueNotifyingHost()`. This function triggers a cascade of synchronized events:

- The APVTS updates its internal value atomically.
- The `SliderAttachment` classes detect the change and visually update the bottom knobs.
- The audio processor (backend) is notified to recalculate the filter coefficients.
- The DAW host (if in plugin mode) logs the change for automation.
- The response curve is automatically redrawn to reflect the new state.

Everything happens with no perceptible latency, giving the user a sense of direct control over the sound. It's an immediate feedback interactivity that is fundamental to real-time creativity.

Gesture Management and Automation

In order for the DAW host to correctly register changes during a drag, two critical methods are implemented:

- `beginGestureForPeak(int peak)`: Notifies the host that an edit of that peak is starting.
- `endGestureForPeak(int peak)`: Notifies that editing has finished

These methods allow advanced automation systems to record the entire sequence of changes as a single, coherent 'sentence,' rather than as thousands of individual events. In a modern DAW, this is reflected as a smooth automation curve that tracks exactly how the user moves the mouse during dragging.



25. Visual Theme System: Skins

To improve versatility and allow the plugin to adapt to different aesthetic preferences and working contexts, a modular system of themes (skins) has been implemented that allows changing the visual appearance of the plugin without affecting any aspect of the audio processing.

Skin System Architecture

The solution is based on a C++ structure called 'Skin' that contains around 30 colors, each assigned to a specific visual element of the plugin. When changing the theme, a different 'Skin' structure is simply referenced, and all GUI components automatically use the new colors in their next paint() cycle.

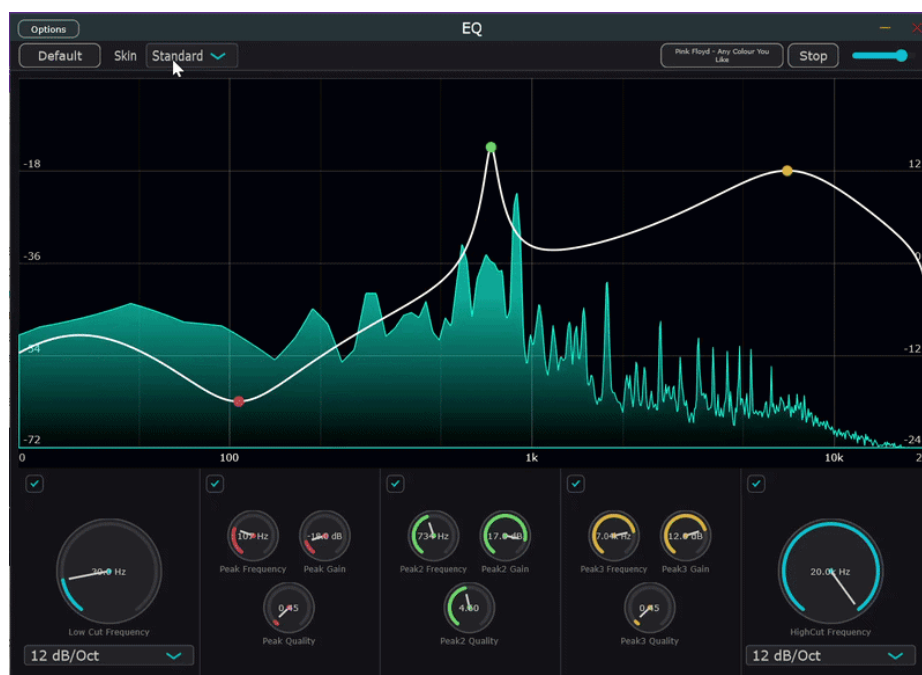
This architecture is extremely efficient because it requires no expensive processing: it's just a pointer change. There's no need to recalculate geometries, resize components, or perform any GPU-intensive operations.

Three Predefined Themes

- Standard (Studio Dark): The default theme. It uses a dark palette (almost black dark gray as the background) with bright cyan accents for controls. It is optimized for studios with controlled lighting and provides a modern, professional look.
- Dark (Vintage Analog): A track that evokes vintage analog equalizers with a dark wood finish, brass details, and warm tones (rust, mustard). The artwork uses warm colors (copper and cream) that mimic the aesthetic of classic instruments from the 70s and 80s.
- Light: A light theme ideal for brightly lit environments or for users who prefer white backgrounds. It uses restrained pastel tones and vibrant Material Design colors for the highlights. It provides high contrast for readability in any lighting condition.

Selecting a Theme from the Interface

A combo box located in the upper right corner of the plugin allows the user to select between the three themes. Changing themes is instantaneous and does not affect the audio parameters or automation. The selection is retained in the session until the user selects a different theme.



26. Integrated File Player:

Release 2 includes a complete audio file playback system that works exclusively in the plugin's standalone mode. This feature allows the user to load and play audio files through the equalizer to evaluate how EQ adjustments affect the sound of the source material in real time.

Supported Audio Formats

The player supports all formats that JUCE can natively decode:

- WAV (uncompressed)
- AIFF (uncompressed)
- MP3 (compressed)
- FLAC (lossless compressed)
- OGG Vorbis (compressed)

Auto-format detection is automatic based on the file extension and its header, so the user simply selects a file and the player handles the appropriate decoder.

File Player Thread-Safe Architecture

The File Player operates on three main components that are synchronized by a `juce::CriticalSection` (`playerLock`):

- `juce::AudioTransportSource`: Transport manager that maintains playback position, playback speed and state (playing/paused/stopped).
- `juce::AudioFormatReaderSource`: A decoder that extracts audio samples from the file. It handles both uncompressed and compressed files transparently.
- `juce::AudioFormatManager`: Decoder recorder that allows JUCE to auto-detect which decoder to use based on the file extension.

Integration into ProcessBlock

Within the `processBlock` method, a `juce::ScopedTryLock` is used to attempt to acquire the lock without ever blocking the audio thread. If the lock is successfully acquired and the player is playing, the next audio block is fetched from the file, the volume gain is applied, and the resulting buffer completely replaces the processor input. The audio from the file then flows through the entire DSP filter chain.

This architecture ensures that even if the user interacts with the 'Load', 'Play', 'Stop' buttons from the graphical interface thread, the audio thread will never be blocked waiting, preventing clicks (glitches) or interruptions in processing.

Interface Controls

Three buttons on the top strip control the player:

- Input Button: Opens a file chooser that allows you to select an audio file. Once loaded, the file name appears on the button.
- Play/Stop Button: Toggles between playback and pause of the loaded file. The button's label changes dynamically depending on the state.
- Volume Slider: A slider that appears only when a file is loaded, allowing you to control the player's mix volume (0..1).